

Intelligent and Communicating Systems, ICS

2nd Year Specialty SIQ G02

Lab Report n°04

Title:

Arduino Communications PWM- Sensors-Actuators-Raspberry (Intro)

Studied by:

First Name: Abderrahmane

Last Name: GRINE

E_mail: ma_grine@esi.dz

I. Theory

1. PWM: Definition

Pulse-Width Modulation (PWM) is a technique used in embedded systems to simulate analog output using digital signals. By controlling the duration a digital signal stays HIGH during a cycle (duty cycle). This allows components like LEDs or motors to behave as if they are receiving a continuous analog voltage, the average power delivered to a component can be adjusted; following this equation:

$$U_{\text{equivalent}} = U_{\text{max}} * \text{Duty Cycle}, \quad \text{Example: } 12V * 0.6 = 7.2V$$

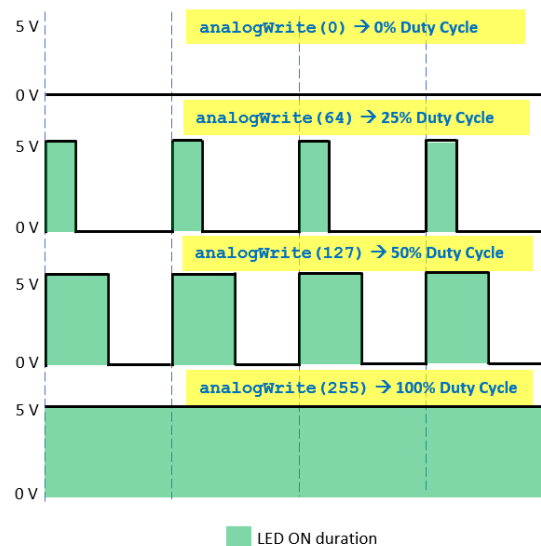


Figure 1.1: Controlling duty cycle with `analogWrite()`¹

2. Arduino vs Raspberry : GPIO, PWM, and Interrupt Pins

2.1. What is Raspberry Pi and How to Install and Configure It

The **Raspberry Pi** is a small, low-cost single-board microprocessor capable of running a full OS (Raspberry Pi OS, Linux-based). Unlike microcontrollers such as Arduino, it is a full computer with CPU, RAM, storage, USB, cameras and networking.

¹Source: wokwi.com. From <https://blog.wokwi.com/explore-pwm-with-logic-analyzer/>

Installation and Configuration Steps:

- Download Raspberry Pi OS from the official website.
- Flash it to an SD card using Balena Etcher.
- Insert SD card, power the Pi and connect it a monitor trough the HDMI port.
- Complete initial setup (language, Wi-Fi, SSH, update packages).
- Configure GPIO access using Python libraries (RPi.GPIO, gpiozero).

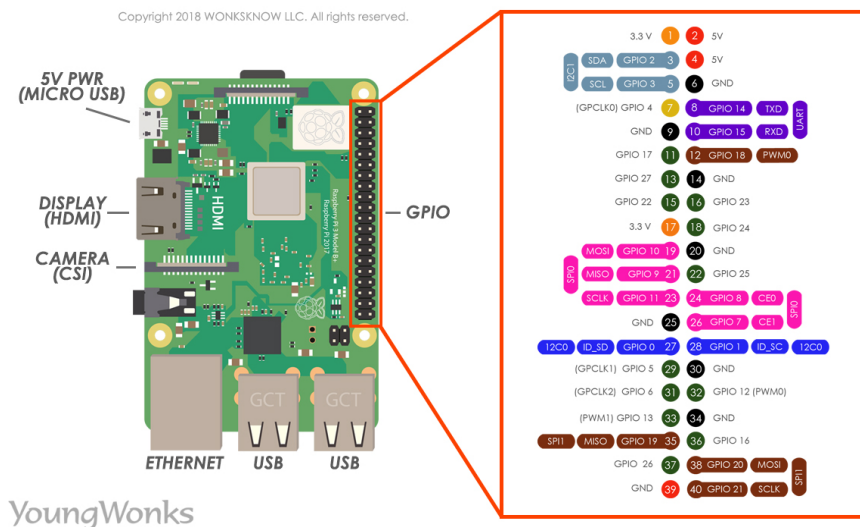


Figure 1.2: Raspberry Pi datatsheet.²

2.2. Comparing Arduino Analog GPIO with A0 and PWM

The **Arduino MKR 1010** provides several types of pins to interface with sensors and actuators. Digital pins (0–8, 10, 12, 18, 19) can be used either as general-purpose inputs/outputs (GPIO) or as PWM outputs to simulate analog voltages by varying the duty cycle of a digital signal. In addition, the board features a true analog output (DAC) on pin A0, allowing precise voltage control from 0 to 3.3V with 10-bit resolution.

Overall, PWM pins are for simulating analog output, and the DAC pin provides true analog output for applications requiring higher precision.

²Source: YoungWonks.com. From <https://www.youngwonks.com/blog/Raspberry-Pi-3-Pinout>

2.3. Theoretical Study of Analog, PWM, and Interrupts of Arduino Pins

Feature	Description	Pins
ADC	Reads 0–3.3V, 10/12-bit	A0–A6
PWM	Digital output, variable duty cycle	0–8, 10, 12, 18, 19
DAC	True analog output, 0–3.3V	A0
Interrupts	Fast response to events	Most digital pins

Table 1.1: Arduino Pin Features

2.4. Theoretical Study of Analog, PWM, and Interrupts of Raspberry Pi Pins

Feature	Description	Pins / Notes
GPIO	Digital only (HIGH/LOW)	All pins
PWM	Output only, simulate analog	GPIO12, 13, 18, 19 (hardware PWM), others via software
Analog I/O	Requires external ADC/DAC	e.g., MCP3008, MCP4725
Interrupts	Detect all edges	All GPIO pins

Table 1.2: Raspberry Pi Pin Features

2.5. Comparing Arduino vs Raspberry GPIO, PWM, and Interrupt Pins

Feature	Arduino MKR 1010	Raspberry Pi
GPIO	Digital + Analog (A0–A6) fewer ,easy to use	Digital only, requires configuration
Analog Input (ADC)	7 pins, 10–12 bit	None (requires external ADC)
Analog Output (DAC)	1 pin A0, 10-bit	None (requires external DAC)
PWM Output	13 pins (0–8, 10, 12, 18, 19)	4 hardware PWM (GPIO12, 13, 18, 19), software PWM others
Interrupts	Limited digital pins	All GPIO pins

Table 1.3: Arduino vs Raspberry Pi Comparison

II. Activity

1. PWM LED Usage

1.1. Hardware Setup

Connect a single LED to the Arduino board through a 220 Ω resistor to ensure a safe current level. The LED is driven using digital pin D9, which supports Pulse Width Modulation (PWM). This setup enables the observation of brightness variations produced by PWM signals.

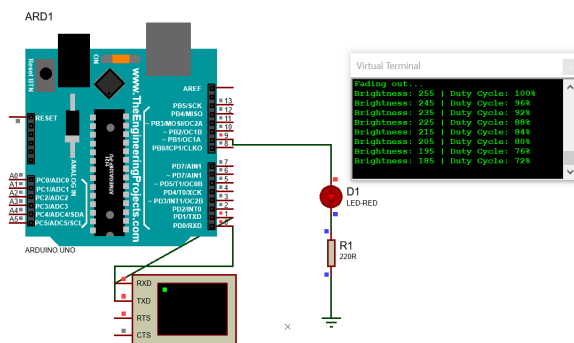


Figure 2.1: PWM-Controlled LED schematic.

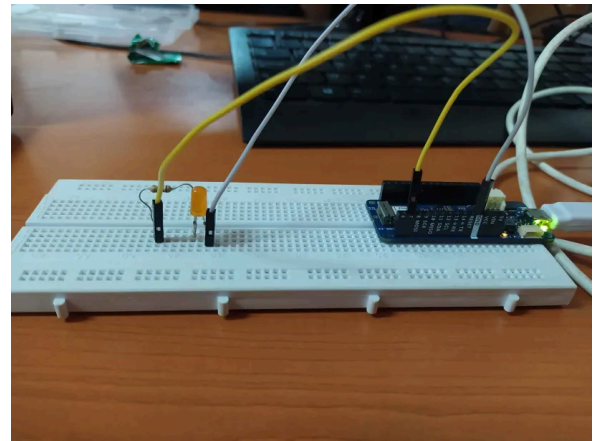


Figure 2.2: PWM-Controlled LED circuit.

1.2. Software Implementation

```
1 const int ledPin = 9; // PWM-capable pin (7 in the circuit)
2
3 void setup() {
4   pinMode(ledPin, OUTPUT); // Configure LED pin as output
5   Serial.begin(9600); // Open serial monitor
6   Serial.println("PWM LED Fade Control Started");
7 }
8
9 void loop() {
10  for (int brightness = 0; brightness <= 255; brightness += 5) {
11    analogWrite(ledPin, brightness); // Write PWM value to pin
12
13    Serial.print("Brightness: ");
14    Serial.print(brightness);
15
16    int duty = map(brightness, 0, 255, 0, 100); //mapping to 100% to
17    simulate duty cycle
18    Serial.print(" | Duty Cycle: ");
19    Serial.print(duty);
```

```
19   Serial.println( "%");
20
21   delay(100);      // Delay to make fading visible
22 }
23 // Fade out from 255 to 0
24 Serial.println( "Fading out ... ");
25 for (int brightness = 255; brightness >= 0; brightness -= 5) {
26     analogWrite(ledPin, brightness);
27
28     Serial.print( "Brightness: ");
29     Serial.print(brightness);
30
31     int duty = map(brightness, 0, 255, 0, 100);
32     Serial.print( " | Duty Cycle: ");
33     Serial.print(duty);
34     Serial.println( "%");
35
36     delay(100);
37 }
38 }
```

Listing 2.1: 8-bit PWM LED Fade In/Out

1.3. Interpretation

The LED's brightness follows the PWM duty cycle, which corresponds to the proportion of time that the signal remains in the HIGH state. Although the voltage is not truly analog, the rapid switching (approximately 490 Hz on Arduino Uno) creates the perception of varying brightness due to the persistence of human vision. The fading effect illustrates the ability of PWM to simulate intermediate brightness levels between fully ON and fully OFF.

2. PWM–LDR–LED Usage

2.1. Hardware Setup

The LDR (Light Dependent Resistor) is connected in a voltage divider configuration, where variations in ambient light result in changes in its resistance. This produces a corresponding analog voltage that is read through analog pin A1. The output of the system is an LED connected to PWM pin D9, allowing the light intensity to adapt automatically depending on the measured ambient illumination.

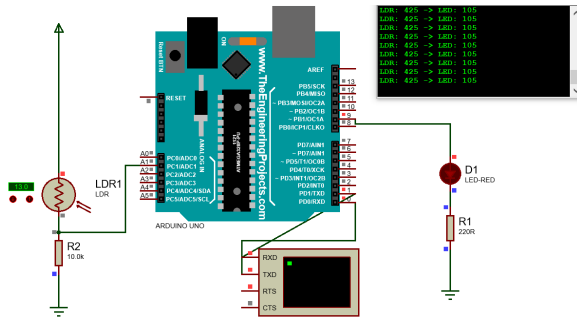


Figure 2.3: LDR-controlled PWM LED schematic.

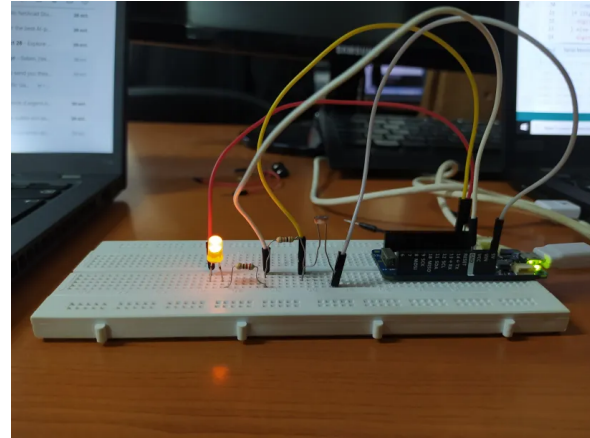


Figure 2.4: LDR-controlled PWM LED circuit.

2.2. Software Implementation

```

1 const int ledPin = 9;    // PWM pin for LED
2 const int ldrPin = A1;  // Analog pin for LDR
3 void setup() {
4     pinMode(ledPin, OUTPUT);    // LED output
5     Serial.begin(9600);          // Start Serial Monitor
6 }
7 void loop() {
8     int ldrValue = analogRead(ldrPin); // Read LDR (0–1023) 10bits
9                                     automatically
10    int ledBrightness = map(ldrValue, 0, 1023, 0, 255); // since PWM is only
11                                     8bit
12    analogWrite(ledPin, ledBrightness); // Set LED brightness
13    Serial.print("LDR: ");
14    Serial.print(ldrValue);
15    Serial.print(" -> LED PWM: "); Serial.println(ledBrightness);
16    delay(50);
17 }

```

Listing 2.2: PWM LED controlled by LDR

2.3. Interpretation

Variations in ambient light are translated into corresponding PWM duty cycle values. Because we use a 10-bit ADC, the input ranges from 0 to 1023, which is sufficient for smooth LED variation.

If the mapping function is not used, the raw LDR value (0–1023) is written directly to `analogWrite()`, which only accepts values between 0 and 255. Any value above 255 causes `analogWrite()` to overflow and wrap around, because only the lower 8 bits are kept. This produces unexpected and incorrect LED behavior:

Values from 0 to 255 behave normally → LED increases in brightness.

Values from 256 to 511 overflow to 0–255 again, causing the LED to dim or turn off unexpectedly. same from 512 to 767 and from 768 to 1023.

So instead of a smooth response, the LED alternates between bright and dim, and sometimes turns off when the ADC value exceeds 255, producing a strange flickering pattern.

3. PWM vs DAC (A0) Comparison

3.1. Hardware Setup

This part uses two LEDs to compare the behavior of PWM output and true analog output using a DAC pin available on Arduino MKR boards. The first LED is connected to digital pin D9 (PWM), while the second is connected to pin A0, which acts as a digital-to-analog converter producing a continuous voltage level. An LDR is again connected to A1 as the analog input source.

Figure 2.5: PWM and DAC circuit.

3.2. Software Implementation

```

1 const int ldrPin = A1;           // LDR input
2 const int pwmLedPin = 9;         // PWM output LED
3 const int analogLedPin = A0;     // DAC output LED
4
5 void setup() {
6   pinMode(pwmLedPin, OUTPUT);
7   pinMode(analogLedPin, OUTPUT);
8   Serial.begin(9600);
9
10  analogReadResolution(12); // Set ADC to 12-bit (0–4095)
11  analogWriteResolution(10); // Set DAC to 10-bit (0–1023)
12 }
13
14 void loop() {
15   int ldrValue = analogRead(ldrPin); // Read 12-bit value (0–4095)
16   Serial.print("LDR (12-bit): ");
17   Serial.println(ldrValue);
18
19   // PWM LED (simulated analog)
20   int pwmBrightness = map(ldrValue, 0, 4095, 0, 255);
21   analogWrite(pwmLedPin, pwmBrightness);
22   Serial.print("PWM LED: ");
23   Serial.println(pwmBrightness);
24
25   // DAC LED (true analog)
26   analogWrite(analogLedPin, ldrValue);
27
28   float analogVoltage = (ldrValue / 1023.0) * 3.3; // Convert to volts
29   Serial.print("DAC LED Voltage: ");

```



```
30 Serial.println(analogVoltage);  
31  
32 delay(100);  
33 }
```

Listing 2.3: PWM vs DAC LED Control

3.3. Interpretation

The PWM LED demonstrates a brightness controlled through the modulation of duty cycle, which is inherently a digital signal. In contrast, the DAC LED receives a smooth voltage varying continuously between 0 and 3.3 V, resulting in noticeably smoother transitions. This highlights the fundamental difference between emulated analog output (PWM) and true analog generation using a DAC.

4. Applications of PWM

PWM is widely used in embedded systems due to its efficiency, precision, and low cost. It allows digital control of devices that normally require analog signals. Some key applications include:

- **Motor Control:** PWM adjusts motor speed accurately without complex analog circuits.
- **EHD Printing Optimization:** PWM is used to shape and control the Electrohydrodynamic (EHD) waveform. By adjusting the pulse width, PWM helps stabilize jet formation and improves printing resolution and material deposition accuracy.¹
- **Fan Speed Regulation:** Ensures smooth (due to duty cycle precision), quiet operation in computers and HVAC (Heating, Ventilation, and Air Conditioning) systems.

5. Conclusion

Through these experiments, PWM has been shown to be a crucial technique for generating analog-like behavior using purely digital hardware. Whether controlling LED brightness or responding to sensor input, PWM provides an efficient and flexible solution that is very suitable for many embedded applications especially when using powerful nano-computers like Raspberry Pi when the cost-effectiveness is a priority. The comparison between PWM and DAC highlights the strengths and limitations of each method, reinforcing the importance of choosing the appropriate output technique depending on the precision, smoothness and budget.

¹See: <https://www.nature.com/articles/s41378-025-00901-x>